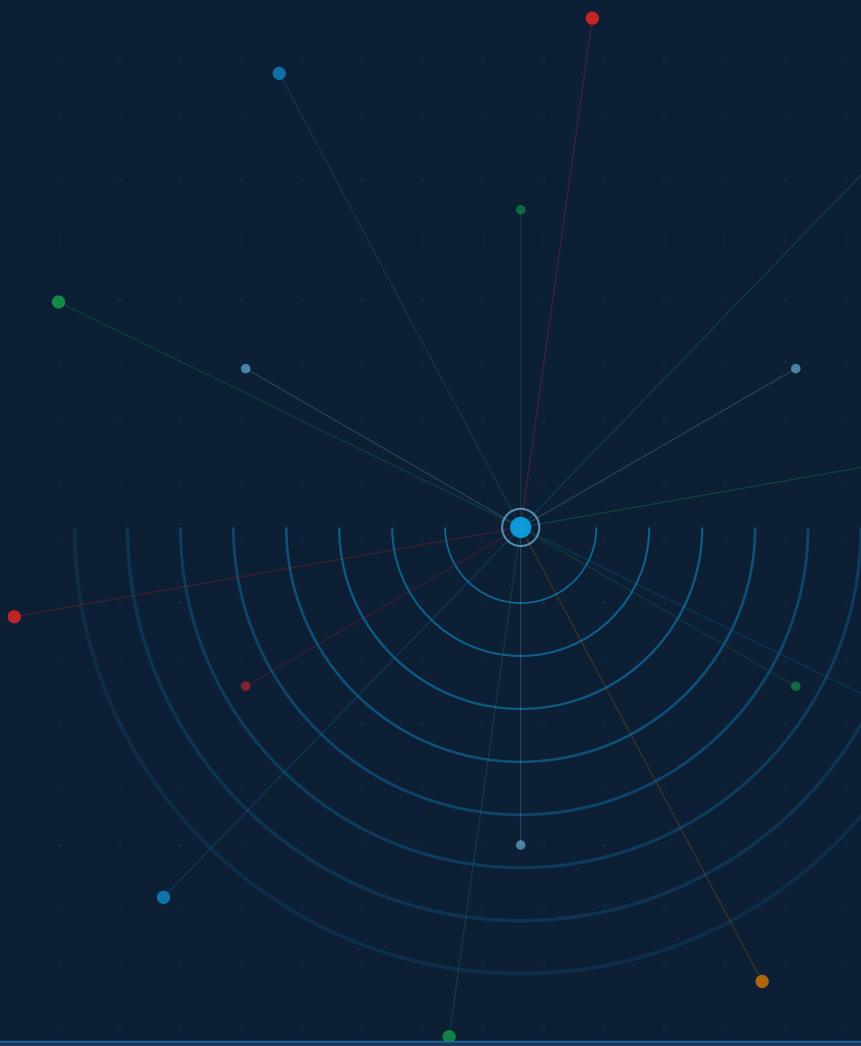
JA4proxy

Enterprise TLS Security Proxy

Operator's User Guide

For Security Engineers and Operations Teams deploying JA4proxy



Version 2.0.0 | Last reviewed 2026-06-10 | github.com/seanpor/JA4proxy

POC — Functional & Tested

JA4proxy Documentation Series

Product Overview	Introduction for buyers and evaluators
Operator's User Guide	**This document** — installation, configuration, day-to-day operations
Technical Reference Manual	Complete configuration, signals, metrics, and API reference

Status: POC — Functional and tested. Not yet production-hardened. See *DMZ Deployment Readiness* in the reference manual for known gaps.

JA4proxy is an open-source project. Source code and issue tracker:

`https://github.com/seanpor/JA4proxy`

JA4 fingerprint specification: `https://github.com/FoxIO-LLC/ja4`

Contents

Preface

This guide is for the engineer who has to make JA4proxy work. Not for the evaluator deciding whether to buy it, and not for the developer reading the source code. For the security engineer or operations team who has been handed a repository URL and told to get it in front of production traffic.

If that is you, this document covers what you need: installing the stack, understanding what the proxy is doing and why, configuring it for your environment, keeping it running, and responding when something goes wrong.

How This Guide Is Organised

The chapters follow the natural sequence of operating JA4proxy for the first time and then day-to-day:

Chapter 1 — Introduction
: What JA4proxy actually does, and the key concepts (TLS fingerprinting, the dial, the pipeline) you need to understand before touching any configuration.

Chapter 2 — Installation
: Getting the stack running from scratch. Prerequisites, step-by-step setup, and how to verify everything is healthy.

Chapter 3 — First Steps
: What to look at once the proxy is running. Generating test traffic, reading the logs, and opening the Grafana dashboard for the first time.

Chapter 4 — Configuration
: The full configuration file walkthrough: the dial, country filtering, fingerprint lists, rate limiting, and bypass conditions.

Chapter 5 — Day-to-Day Operations
: Starting and stopping, adding entries to lists, managing bans, updating the GeoIP database.

Chapter 6 — Monitoring
: The Grafana dashboard in detail, key Prometheus metrics, Loki log queries, and pre-configured alerts.

Chapter 7 — Incident Response
: Detecting an active attack, emergency blocking procedures, investigating suspicious fingerprints, recovering from false positives.

Chapter 8 — Scaling
: Adding proxy instances, what state is shared, Kubernetes deployment and autoscaling.

Chapter 9 — Troubleshooting
: Common failure modes and how to fix them. Diagnostic commands and log analysis.

Quick Reference
: All `make` targets, Redis CLI commands, port numbers, and metric names on one page.

The Technical Reference Manual

This guide deliberately stops short of exhaustive technical detail. Signal definitions, full metric listings, Redis key schemas, API specifications, and architecture decision records live in the companion *Technical Reference Manual*. Where this guide says *see the Reference Manual*, that is where to look.

Status Note

JA4proxy is a proof-of-concept: functional, tested against real traffic profiles, and used in pre-production environments. It is *not* yet production-hardened in the sense that an audited commercial product would be. Secrets management, TLS mutual authentication between internal services, and several other hardening items are documented as known gaps in the Reference Manual's *DMZ Deployment Readiness* chapter. Read that chapter before placing JA4proxy in front of live user traffic.

This guide is honest about those limitations. Where something requires additional hardening before production, it says so plainly.

Conventions

`monospace`	Commands, file paths, configuration keys, and code.
`config.key`	Configuration file keys in `config/proxy.yml`.
Side notes	Supplementary information in the margin.[1]
Note boxes	Provide important information you should read.

[1] Margin notes like this one provide context or cross-references without interrupting the main text.

Warning boxes	Flag things that can cause operational problems.
Danger boxes	Describe actions that can cause data loss or outages.

Example note box

This is a note box. Use these for important context that does not interrupt the main flow.

Example warning box

This is a warning box. Use these for operational risks.

Seán Ó Ríordáin
JA4proxy Project

Introduction

Most TLS security products work by breaking encryption. They sit between the client and the server, decrypt the traffic, inspect it, re-encrypt it, and forward it on. This gives them visibility into HTTP content, but it comes at significant cost: the product must hold your TLS private keys, your users are exposed to an additional certificate chain they did not negotiate, and performance overhead is substantial.

JA4proxy takes a different approach. It never decrypts anything.

The Core Insight: TLS Fingerprinting

The TLS handshake begins with a *ClientHello* message sent in plaintext before any encryption is established. This message contains a structured description of the client's TLS capabilities: which protocol versions it supports, which cipher suites it offers, which extensions it uses, and in what order.

The combination of these fields is highly characteristic of the software that generated them. Chrome 120 on Windows produces a different ClientHello than curl 8.3, which produces a different ClientHello than a Cobalt Strike beacon, which produces a different ClientHello than an automated credential-stuffing tool.

The *JA4* algorithm[2] hashes these ClientHello fields into a compact, human-readable fingerprint string. A typical fingerprint looks like `t13d1113h2_8daaf6152771_b0da82dd1658`. The prefix encodes the TLS version, the middle section encodes the cipher and extension set, and the suffix encodes the ALPN values.

[2] JA4 is a fingerprinting algorithm by FoxIO. Specification at https://github.com/FoxIO-LLC/ja4

JA4proxy computes this fingerprint for every incoming connection — before the handshake completes, before any HTTP request is made — and uses it as the primary signal for traffic classification.

The architectural guarantee

Because JA4proxy reads only the ClientHello plaintext, it never holds TLS private keys and never terminates TLS. The encrypted session passes through unchanged. Your backend server termi-

> nates TLS exactly as it would without JA4proxy in the path.

What JA4proxy Does Not Do

Understanding the boundaries is as important as understanding the capabilities.

JA4proxy *does not*:

- Inspect HTTP request content, headers, or bodies
- Decrypt TLS traffic
- Parse application-layer protocols
- Replace a WAF for content-based attack detection (SQLi, XSS, and similar)
- Authenticate individual users

JA4proxy *does*:

- Identify and block known-bad TLS fingerprints (C2 tools, crawlers, bots)
- Enforce minimum TLS version and cipher suite requirements
- Detect and rate-limit automated scanning behaviour
- Block connections from known-bad IP addresses, ASNs, and countries
- Identify Tor exit nodes, datacenter IP ranges, and VPN providers
- Detect beaconing patterns (regular-interval connections characteristic of C2 traffic)
- Apply reputation data from AbuseIPDB and Spamhaus DROP lists

The Network Architecture

JA4proxy sits between your load balancer and your backend. In the reference deployment:

```
Internet
  |
  v  :443 (TLS, PROXY protocol v2)
HAProxy (load balancer)
  |
  v  :8080 (TCP passthrough)
JA4proxy x N  <----> Redis (shared state)
  |
  v  :443 (TLS, unmodified)
Backend server
```

HAProxy terminates nothing — it operates in TCP passthrough mode, forwarding the raw TLS stream to JA4proxy with a PROXY protocol v2 header prepended so the proxy knows the real client IP

address. JA4proxy reads the ClientHello, makes its decision, and either forwards the connection to the backend unchanged or closes it.

Multiple JA4proxy instances share state through Redis: the JA4 blacklist and whitelist, IP bans, rate-limiting windows, and enrichment results are all available to every instance.[3]

[3] This means you can scale to many proxy instances without any sticky-session requirement.

The Decision Pipeline

Every connection passes through a two-stage pipeline.

Stage 1: Fast-Path Bypasses

Before any scoring occurs, a set of conditions can short-circuit the pipeline entirely. These are *bypass checks* — they produce an immediate ALLOW or BLOCK decision without consulting the scorer.

Condition	Behaviour	Configurable
h2/h1 ALPN (browser)	ALLOW immediately	Yes
JA4 in whitelist	ALLOW immediately	Yes
Valid mTLS client cert	ALLOW immediately	Yes
IP in static allowlist	ALLOW immediately	Yes
JA4 in blacklist	BLOCK immediately (RST)	Yes
Country in blocklist	BLOCK immediately	Yes
Spamhaus DROP match	BLOCK immediately	Yes
TLS 1.0/1.1 version	BLOCK immediately (RST)	Yes

Table 1: Fast-path bypass conditions and their defaults

The browser ALPN bypass deserves special mention. Any connection advertising `h2` or `h1` (standard browser ALPN values) is allowed immediately, regardless of any other rule. This is an architectural guarantee against false positives — legitimate browser traffic can never be blocked by JA4proxy, regardless of configuration.[4]

[4] This guarantee exists because the cost of blocking a real user is always higher than the cost of letting through a bot that happens to advertise browser ALPN values. The proxy is designed around this asymmetry.

Stage 2: Signal Collection and Scoring

Connections that reach Stage 2 are analysed by a set of signal collectors running in parallel. Each collector examines one aspect of the connection and produces a *RiskSignal* — a numeric score contribution and a label. The signals are:

The composite scorer aggregates all RiskSignals into a single score from 0 to 100. The action decider then maps that score to a decision based on the current *dial* setting.

The Dial

The dial is a single number from 0 to 100 that controls how aggressively JA4proxy acts on risk scores.

`dial=0` *(monitor only)* The proxy scores all traffic but takes no blocking action. Every connection is allowed. Events are

Signal	What it examines
TLS enforcer	TLS version, cipher strength
SNI analyser	Missing SNI, IP-literal, DGA-pattern hostnames
TCP analyser	JA4T fingerprint, connection lifespan, concurrency
ASN classifier	Datacenter, Tor exit, VPN ranges
FCrDNS enrichment	Forward-confirmed reverse DNS; residential classification
Blocklist check	Spamhaus DROP/EDROP match (as signal, not bypass)
Beaconing detector	IAT coefficient of variation; regular-interval connections
AbuseIPDB	IP reputation score from community abuse reports
RDAP enrichment	Org-level reputation; netblock classification
Analytics findings	Cross-instance campaign and slow-scan detection

Table 2: Signal collectors in Stage 2

logged and metrics are recorded. This is the default and the correct setting for initial deployment.

`dial=100` *(full blocking)* The proxy applies its configured thresholds. Connections with scores above the block threshold are blocked; those above the tarpit threshold are tarpitted; those above the rate-limit threshold are rate-limited.

Why the default is zero

The default `dial=0` is deliberate. On first deployment, you do not yet know what your legitimate traffic looks like. Deploying with blocking enabled risks immediately breaking real users. Spend time at dial=0 reviewing the logs and Grafana dashboard. Only raise the dial when you are confident the scoring matches reality.

Values between 0 and 100 proportionally raise the thresholds at which each action triggers. The dial is designed to be raised gradually: from 0 to 25 (flag only), then to 50 (rate limiting), then to 75 (tarpit high-risk), then to 100 (full blocking). *See Chapter for the full dial rollout procedure.*

Self-Healing Design

Every ban in JA4proxy has a 5-minute TTL.[5] There are no permanent bans by default. The system is designed to fail open: if Redis is unreachable, the proxy allows connections rather than blocking them.

This approach accepts a small number of false negatives (bad traffic slipping through during a recovery window) in exchange for a strong guarantee against false positives (real users being locked out).

[5] This is intentional and cannot be configured away. The rationale: a 5-minute ban on a scanner costs them the same operational disruption as a permanent ban, because scanners retry anyway. But a 5-minute window limits the blast radius when a legitimate IP is incorrectly flagged.

What This Guide Covers

This guide is the operational reference: how to install, configure, operate, monitor, and troubleshoot JA4proxy. It assumes you have a working knowledge of Docker, TLS basics, and network security concepts.

For architecture decision records, signal algorithm specifications, full Prometheus metric definitions, Redis schema documentation, and API references, see the *Technical Reference Manual*.

Installation

JA4proxy runs as a Docker Compose stack. There is no native package installation and no binary release to configure by hand. Everything — the proxy instances, Redis, HAProxy, the analytics node, and the observability stack — runs in containers managed by Compose.

This chapter covers getting that stack running from a clean machine. It takes approximately 10–15 minutes.

Prerequisites

Software

Requirement	Minimum version	Notes
Docker Engine	24.0	Docker Desktop on macOS/Windows works
Docker Compose	v2.20	Bundled with recent Docker installations
GNU Make	3.81	Pre-installed on Linux and macOS
Git	Any	For cloning the repository

Table 3: Software prerequisites

Verify your installed versions:

```
docker --version
docker compose version
make --version
```

Ports

The following ports must be available on the host machine. If any are in use by another service, stop that service or adjust the port mapping in `docker-compose.yml` before proceeding.

Firewall note

In production, only port 443 should be exposed to the public internet. All other ports should be restricted to your management network or accessible only via VPN. The development stack exposes all ports to localhost for convenience.

Table 4: Port assignments

Port	Service	Purpose
443	HAProxy	Public TLS entry point
8080	JA4proxy	Direct proxy access (internal)
8404	HAProxy stats	HAProxy statistics page
8443	Backend	Test backend HTTPS server
8888	Tarpit	Slow-response tarpit service
9090	JA4proxy metrics	Prometheus scrape endpoint (proxy)
9091	Prometheus	Prometheus query UI
3001	Grafana	Dashboard UI
3100	Loki	Log aggregation
9093	Alertmanager	Alert routing

System Resources

The full stack (proxy, Redis, analytics, observability) requires:

- 4 GB RAM available to Docker
- 2 CPU cores
- 5 GB disk (images + GeoIP databases + logs)

On macOS and Windows, ensure Docker Desktop is configured with at least these resources in its settings.

Step-by-Step Installation

Step 1: Clone the Repository

```
git clone https://github.com/seanpor/JA4proxy.git
cd JA4proxy
```

Step 2: Create the Environment File

JA4proxy uses a `.env` file for deployment-specific configuration that you do not want to commit to version control (credentials, backend hostnames, API keys).

```
cp .env.example .env
```

Open `.env` in your editor and set at minimum:

```
# The hostname or IP address of your backend server
BACKEND_HOST=your.backend.host

# Grafana admin credentials
GRAFANA_ADMIN_USER=admin
GRAFANA_ADMIN_PASSWORD=changeme

# AbuseIPDB API key (optional - features degrade gracefully
    without it)
ABUSEIPDB_API_KEY=

# IP2Location API key for GeoIP database download
```

```
# Register free at https://lite.ip2location.com
IP2LOCATION_TOKEN=
```

Change default credentials

The default Grafana credentials in `.env.example` are public. Change them before running the stack, even in a development environment.

Step 3: Start the Stack

```
make start
```

This builds all Docker images and starts the full stack. On a fast connection, the first build takes 3–5 minutes as it pulls base images. Subsequent starts are much faster.

You will see Docker Compose output as each container starts. When all containers report healthy status, the stack is ready.

Step 4: Download the GeoIP Database

Country-based filtering requires the IP2Location Lite country database. Download and install it:

```
make update-geoip
```

This downloads the database from IP2Location using the token in your `.env` file and places it where the proxy can read it. If `IP2LOCATION_TOKEN` is not set, the command will print instructions for manual download.

Free tier is sufficient

The IP2Location Lite database is free for non-commercial use. Country-level accuracy is what JA4proxy needs — the free tier provides this. Register at `https://lite.ip2location.com` and copy your token to `.env`.

Country filtering is disabled by default in `config/proxy.yml` until the database is present. The proxy logs a warning at startup if GeoIP is enabled but the database file is missing.

Verifying the Installation

Container Status

Check that all containers are running and healthy:

```
docker compose ps
```

Expected output shows all services in the `running` state. The proxy and Redis containers include health checks — wait for them to show `(healthy)` before proceeding.

Proxy Health Endpoint

```
curl -s http://localhost:8080/health | python3 -m json.tool
```

A healthy response:

```
{
  "status": "healthy",
  "redis": "connected",
  "geoip": "loaded",
  "uptime_seconds": 42
}
```

HAProxy Statistics

Open `http://localhost:8404/stats` in a browser. You should see the HAProxy statistics page showing the `ja4proxy_backend` farm with at least one server in UP state.

Prometheus Scrape Targets

Open `http://localhost:9091/targets`. All targets should show `UP` state. If the `ja4proxy` target is down, the proxy metrics endpoint is not responding — check the proxy container logs.

Grafana Dashboard

Open `http://localhost:3001` and log in with your Grafana credentials. The *JA4proxy Overview* dashboard should be available under Dashboards. At this point it will show zero connections — that is expected.

Generate test traffic

To see the dashboard populate immediately, generate some test traffic:

```
./scripts/generate-tls-traffic.sh 30 50 5
```

This runs for 30 seconds with 50% legitimate traffic and 5 worker threads. See Chapter for a full explanation of the traffic generator.

Rebuilding the Stack

If you need to rebuild all images from scratch (after pulling a new version of the repository, or if the images are in an inconsistent state):

```
make rebuild
```

Wipes all data

`make rebuild` stops all containers, removes images, and rebuilds from scratch. Any data in Redis (bans, rate-limit windows) will be lost. Use this only when you need a clean slate or after updating the repository.

Building the Go Proxy (Optional)

A Go implementation of the proxy is included in `proxy_go/`. It is currently in active development targeting 10–50× the throughput of the Python implementation. The Python proxy is the production-stable version; the Go proxy is for evaluation and performance testing.

To build the Go proxy:

```
cd proxy_go
go build -o ja4proxy ./cmd/proxy
./ja4proxy --config ../config/proxy.yml
```

Go proxy status

The Go proxy achieves feature parity with the Python proxy for all core pipeline functions. Some auxiliary features (management API, analytics integration) are still being finalized. See `docs/phases/PHASE_15.md` for the current parity status.

Stopping the Stack

To stop without removing data:

```
make stop
```

To stop and remove all containers, networks, and volumes (clean slate):

```
make stop-clean
```

`make stop` leaves Redis data and logs intact. `make stop-clean` removes everything including Redis persistence.

First Steps

The proxy is running and the dashboard is open. Now what?

This chapter walks through what you will see in the first minutes of operation: what the logs mean, how to generate representative test traffic, and how to read the Grafana dashboard. By the end of this chapter you should be able to tell from the dashboard whether JA4proxy is classifying traffic correctly.

What Happens at Dial Zero

At `dial=0` (the default), JA4proxy is in monitor mode. Every connection is analysed, scored, and logged — but no connection is ever blocked. The proxy passes everything through to your backend regardless of its risk score.

This is the right mode for initial deployment. You are building a baseline understanding of what your traffic looks like before you start acting on it.

Logs will show entries like this:

```
ALLOWED:  172.19.0.10 | Country: IE | JA4:
    t13d1113h2_8daaf6152771_b0da82dd1658 |
         Name: Browser (TLS 1.3) | TLS: TLS 1.3
ALLOWED:  185.220.0.1 | Country: RU | JA4:
    t13d0912_a1b2c3d4e5f6_987654321abc |
         Name: Tool/Bot (TLS 1.3) | TLS: TLS 1.3 | Score: 74
```

At dial=0, both lines show ALLOWED. The second entry has a high risk score (74) and would be blocked at dial=100 — but in monitor mode it passes through. This is correct behaviour. You are observing, not acting.[6]

[6] The score is still computed and recorded even at dial=0. This means when you eventually raise the dial, you already have weeks of score history to reason about.

The Traffic Generator

JA4proxy ships with a traffic generator for testing and demonstration. It sends realistic TLS connections using a mix of legitimate browser profiles and known-malicious tool profiles.[7]

[7] Profiles included: Chrome, Firefox, Safari (legitimate) and Sliver C2, Cobalt Strike, Evilginx, Python bot, credential stuffer (malicious).

```
./scripts/generate-tls-traffic.sh <duration_seconds> <
    legit_percent> <workers>
```

For example, to run for 60 seconds with 30% legitimate traffic and 10 workers:

```
./scripts/generate-tls-traffic.sh 60 30 10
```

A good first run to populate the dashboard:

```
# 60 seconds, mixed traffic, moderate load
./scripts/generate-tls-traffic.sh 60 50 20
```

While it runs you will see log output showing connections being processed. Some will have fingerprints identified as known tools; these will show elevated risk scores. With dial=0 they will all be allowed, but the scoring information appears in the log.

Reading the Logs

Log Line Format

Each connection produces one log line on close. The format is:

```
{ACTION}: {client_ip} | Country: {cc} | JA4: {fingerprint} |
          Name: {label} | TLS: {version} [| Reason: {reason}]
          [| Score: {score}]
```

Table 5: Log line fields

Field	Meaning
`ACTION`	`ALLOWED`, `BLOCKED`, `TARPITTED`, or `RATE_LIMITED`
`client_ip`	Real client IP (extracted from PROXY protocol header)
`Country`	Two-letter country code from GeoIP lookup
`JA4`	The computed JA4 fingerprint string
`Name`	Human-readable label for the fingerprint, if known
`TLS`	TLS version negotiated in the ClientHello
`Reason`	For blocked connections, the blocking reason
`Score`	Risk score (0–100); appears when score is non-zero

Example Log Lines

A legitimate browser connecting from Ireland:

```
ALLOWED:  172.19.0.10 | Country: IE | JA4:
    t13d1113h2_8daaf6152771_b0da82dd1658 |
          Name: Browser (TLS 1.3) | TLS: TLS 1.3
```

A known tool blocked by the JA4 blacklist:

```
BLOCKED:  185.220.0.1 | Country: RU | JA4:
    t13d0912_a1b2c3d4e5f6_987654321abc |
          Name: Tool/Bot (TLS 1.3) | Reason: JA4 blacklisted
```

An IP that has been banned (reached rate limit and been automatically banned):

```
BLOCKED:  10.0.0.42 | Country: DE | JA4:
    t13d1113h2_8daaf6152771_b0da82dd1658 |
          Name: Browser (TLS 1.3) | Reason: Banned for 300s
```

A high-scoring connection at dial=0 (passed through but scored):

```
ALLOWED:  198.51.100.3 | Country: NL | JA4:
    t13d0809_c0ffee123456_aabbccddeeff |
          Name: Python-requests (TLS 1.3) | TLS: TLS 1.3 |
    Score: 68
```

Accessing Logs

Live tail all proxy output:

```
docker compose logs -f ja4proxy
```

Filter for blocked connections only:

```
docker compose logs ja4proxy | grep "^BLOCKED"
```

Search for a specific IP:

```
docker compose logs ja4proxy | grep "185.220.0.1"
```

The Grafana Dashboard

Open `http://localhost:3001` and navigate to the *JA4proxy Overview* dashboard. After running the traffic generator you should see all panels populated.

Panel-by-Panel Guide

Connection Rate

Shows connections per second over time, coloured by action:

- Green: ALLOW — connections passed to backend
- Red: BLOCK — connections rejected
- Amber: TARPIT — connections held in tarpit
- Dark: BAN — connections from currently-banned IPs

At dial=0, everything is green. When you raise the dial, you will start to see red and amber segments appear.

Action Distribution

A pie or bar chart showing the percentage breakdown of actions over the selected time window. This is the key panel for understanding how JA4proxy is classifying your traffic. In a healthy deployment you should see the vast majority as ALLOWED, with a small but non-zero fraction as BLOCKED.

If you see zero BLOCKED and you know malicious traffic is present, the dial is probably at zero. If you see a high BLOCKED fraction, investigate before raising the dial further — high block rates often mean a fingerprint list needs tuning.

Risk Score Distribution

A histogram of risk scores across all connections. This is your most important panel for dial calibration. The shape of this histogram tells you where to set blocking thresholds:

- A bimodal distribution (peaks near 0 and near 80–100) is ideal: legitimate traffic clusters at low scores, bad traffic clusters at high scores.
- A single peak near zero means either the traffic is mostly legitimate, or the signal collectors are not seeing the bad traffic you expect.
- A broad flat distribution suggests mixed signals — worth investigating which specific signals are contributing.

Top JA4 Fingerprints

A table of the most frequent JA4 fingerprints seen, with their labels (if known), connection counts, and average risk scores. This is where you identify candidate fingerprints for your whitelist or blacklist.

If a fingerprint appears frequently with a high average score but a human-readable label that sounds legitimate, investigate before blacklisting it — it may be a shared fingerprint used by both automated and manual clients.

Geographic Distribution

A world map or table showing connection volume by country. Useful for identifying whether a sudden traffic increase is concentrated in an unexpected region.

Rate Limiting Activity

Shows how many connections are hitting each rate-limit strategy (by IP, by JA4 fingerprint, by IP+JA4 pair). High activity here means automated clients are testing your rate limits.

Redis Lag

Shows Redis response time. Spikes here affect scoring latency. If Redis lag is consistently above 1ms, investigate the Redis container's resource usage.

Understanding Fingerprint Labels

The *Name* field in logs and the label column in the Grafana fingerprint table come from `config/proxy.yml`'s `fingerprint_labels` section. This is a mapping from JA4 fingerprint strings to human-readable names.

JA4proxy ships with labels for common browsers, known security tools, and common C2 frameworks. You will extend this list as you observe your own traffic.

When you see an unfamiliar fingerprint appearing frequently, look it up in the JA4 community database at `https://ja4db.com` and add an appropriate label to your config. This makes the dashboard and logs much easier to interpret.

Your First Configuration Change

The two most common initial configuration tasks are:

Setting Your Backend Host

If you have not already set `BACKEND_HOST` in `.env`, the proxy is forwarding to the default test backend. Update `.env` and restart:

```
# In .env:
BACKEND_HOST=prod.internal.example.com

make stop
make start
```

Enabling Country Filtering

To restrict traffic to specific countries, edit `config/proxy.yml`:

```
# Enable the whitelist (only listed countries pass)
country_whitelist_enabled: true
country_whitelist:
  - IE
  - GB
  - US
  - DE
```

Then reload the config without restarting (see Section):

```
# Signal the proxy to reload config
```

```
kill -HUP $(docker compose exec ja4proxy cat /var/run/proxy.pid
    )
```

Country filtering and false positives

Country whitelisting is a blunt instrument. VPN users, Tor exit nodes, and CDN origin IPs may appear to come from unexpected countries. Enable country filtering only if you have a genuine operational reason to restrict by geography, and verify your legitimate user base is not affected before enabling.

Configuration

All runtime behaviour is controlled by `config/proxy.yml`. This chapter walks through every significant configuration section, explains what each setting does, and gives guidance on choosing values for your environment.

The configuration file uses YAML. Every key is documented with an inline comment in the file itself; this chapter provides the operational context that comments cannot.

Hot Reload

Configuration changes take effect without restarting the proxy. Send `SIGHUP` to the proxy process:

```
kill -HUP $(docker compose exec ja4proxy cat /var/run/proxy.pid
    )
```

The proxy logs a confirmation:

```
INFO | config | event=reload | status=ok | keys_changed=3
```

The Management UI (when present) triggers reload via a Redis pub/sub message — the proxy subscribes to a reload channel and applies new configuration on receipt.

Configuration sections that *cannot* be hot-reloaded and require a full restart: listen port, Redis URL, TLS certificate paths, and the number of worker processes. These are logged as `restart_required` in the reload response.

The Dial

```
# Blocking aggression: 0 = monitor only, 100 = full blocking
# Default: 0 (never block on first deploy)
dial: 0
```

The dial is the single most consequential configuration setting. Get it wrong and you either block legitimate users or provide no protection.

Dial Rollout Procedure

Follow this progression. Spend at least 24–48 hours at each stage before proceeding, and watch the Grafana action distribution and false-positive indicators throughout.

Dial	Effect	What to watch for
0	Monitor only	Baseline. All traffic passes. Review score distribution.
25	Flag only	High-scoring traffic is flagged in metrics. No blocking.
50	Rate limiting active	Automated clients start hitting rate limits. Check for false positives in rate-limited IPs.
75	Tarpit + rate limit	High-risk connections enter tarpit. Verify no legitimate API clients are affected.
100	Full blocking	Block threshold applies. Last check: review BLOCKED log lines for any unexpected legitimate traffic.

Table 6: Dial rollout stages

Do not jump to dial=100

The dial increment limit exists for a reason. A jump from 0 to 100 on a production system, without first reviewing scores at intermediate stages, risks immediately blocking legitimate users. Always progress gradually.

Dial and Bypass Interaction

ALLOW bypasses (browser ALPN, JA4 whitelist, mTLS) are *unaffected by the dial*. They always allow. Disabling a bypass is the correct way to route that category through the scorer; the dial then controls whether scoring results in a block.

BLOCK bypasses (JA4 blacklist, country block, Spamhaus) also operate independently of the dial at their default setting — they produce hard blocks before scoring. When a BLOCK bypass is disabled, those connections enter the scorer, and the dial then controls what happens to them.

Country Filtering

```
# Country whitelist: only these countries are allowed
# Requires GeoIP database (make update-geoip)
country_whitelist_enabled: false
country_whitelist:
  - IE
  - GB
  - US
```

```yaml

# Country blacklist: these countries are always blocked
# Takes effect only when country_whitelist_enabled is false
country_blacklist_enabled: false
country_blacklist:
  - KP
  - RU
  - CN
```

Whitelist and blacklist are mutually exclusive: if `country_whitelist_enabled` is `true`, the blacklist is ignored (the whitelist already excludes everything not listed). If both are disabled, country filtering has no effect.

Whitelist vs blacklist

The whitelist is the more robust option for deployments where you know your legitimate audience is geographically bounded. The blacklist is more convenient but less effective — attackers are not constrained to operate from the countries you have blocked.
VPN usage means country codes are unreliable. Do not rely on country filtering as your primary defence.

JA4 Fingerprint Lists

```yaml
security:
  # Exact-match whitelist: these fingerprints are always allowed (bypass scoring)
  whitelist:
    - t13d1113h2_8daaf6152771_b0da82dd1658  # Chrome 120 on Windows
    - t13d1113h2_8daaf6152771_02713d6af862  # Chrome 120 on macOS

  # Pattern-match whitelist: regex patterns matched against fingerprint strings
  # Use for version-resilient matching when the full fingerprint changes with browser updates
  whitelist_patterns:
    - "t13d111[0-9]h2_.*"  # Any Chrome-like TLS 1.3 fingerprint

  # Exact-match blacklist: these fingerprints are immediately blocked (hard block)
  blacklist:
    - t13d0912_a1b2c3d4e5f6_987654321abc  # Sliver C2
    - t12d1302_c0ffee123456_aabbccddeeff  # Cobalt Strike default

  # Human-readable labels for known fingerprints (used in logs and dashboard)
  fingerprint_labels:
    t13d1113h2_8daaf6152771_b0da82dd1658: "Chrome 120 (Windows)
```

```
      "
    t13d1113h2_8daaf6152771_02713d6af862: "Chrome 120 (macOS)"
    t13d0912_a1b2c3d4e5f6_987654321abc: "Sliver C2"
```

Managing the Whitelist

Add fingerprints to the whitelist when:

- A legitimate API client or monitoring tool is being scored too harshly
- An internal service has a non-browser TLS fingerprint you know is safe
- A specific customer's tooling is repeatedly triggering rate limits

Use `whitelist_patterns` for browser fingerprints that change with each browser update, to avoid constant maintenance. The pattern `t13d111[0-9]h2_.*` matches all TLS 1.3 connections with Chrome-like cipher ordering, regardless of the specific extension hashes.

Whitelist patterns and performance

Patterns are evaluated as regular expressions for each connection. Keep the pattern list short (under 20 entries) and anchor patterns where possible to avoid backtracking. Exact matches in `whitelist` are O(1) hash lookups.

Managing the Blacklist

Add fingerprints to the blacklist when you have high confidence that a fingerprint represents a malicious tool and that no legitimate software shares it. The JA4 community database at `https://ja4db.com` is a useful reference for known tool fingerprints.

The blacklist produces an immediate RST (hard block) before scoring. This is the most efficient way to drop known-bad clients.

Shared fingerprints

Some penetration testing tools use the same underlying TLS library as legitimate software. Before blacklisting a fingerprint, verify it is not shared with any legitimate client in your environment. Fingerprints from generic HTTP libraries (like Python's `requests` or Node.js's `https`) are often shared.

Rate Limiting

```yaml
rate_limiting:
  enabled: true

  # Three independent strategies - any, all, or a majority can
    trigger
```

```
  rate_limit_strategies:
    by_ip:
      enabled: true
      limit: 60          # connections per window
      window_seconds: 60
      action: ban        # ban, block, or tarpit

    by_ja4:
      enabled: true
      limit: 200         # same fingerprint across all IPs
      window_seconds: 60
      action: tarpit

    by_ip_ja4_pair:
      enabled: true
      limit: 30          # same IP + same fingerprint
      window_seconds: 60
      action: ban

  # How to combine strategy decisions when multiple trigger
  # majority: trigger if >=2 of 3 strategies say yes (default)
  # any: trigger if any strategy says yes (aggressive)
  # all: trigger if all enabled strategies say yes (
    conservative)
  multi_strategy_policy: majority
```

Strategy Selection

The three strategies catch different attack patterns:

`by_ip` Catches any single source sending high volume, regardless of tool. The bluntest instrument — catches bots, scanners, and DDoS sources. Low false-positive risk for normal users.

`by_ja4` Catches distributed attacks using the same tool across many source IPs. A botnet of 1000 IPs each sending 2 connections might not trigger the IP strategy, but will trigger the JA4 strategy if the fingerprint is consistent across the botnet.

`by_ip_ja4_pair` The most precise strategy. Catches a single source using a single tool intensively, while allowing the same IP to use multiple different tools (e.g., a security researcher running multiple scanners from one IP).

Multi-Strategy Policy

`multi_strategy_policy: majority` is the default and recommended setting. It requires two of the three strategies to agree before triggering an action. This reduces false positives from any single strategy misfiring, while still catching distributed attacks that trigger multiple

strategies.

Use `any` only if you need maximum sensitivity and are comfortable with a higher false-positive rate. Use `all` only if you have persistent false positive issues with the majority policy.

Bypass Conditions

```
security_policy:
  # ALLOW bypasses - short-circuit scoring
  alpn_browser_bypass:
    enabled: true        # h2/h1 ALPN -> always allowed

  ja4_whitelist_bypass:
    enabled: true        # JA4 in whitelist -> always allowed

  mtls_bypass:
    enabled: true        # Valid mTLS client cert -> always
    allowed

  static_ip_allowlist:
    enabled: true        # IP in static allowlist -> always
    allowed

  # BLOCK bypasses - hard block before scoring
  ja4_blacklist_bypass:
    enabled: true        # JA4 in blacklist -> immediate RST

  country_blacklist_bypass:
    enabled: true        # Country in blocklist -> immediate
    block

  spamhaus_bypass:
    enabled: true        # Spamhaus DROP/EDROP match ->
    immediate block

  tls_version_bypass:
    enabled: true        # TLS 1.0/1.1 -> immediate RST
```

Every bypass condition is independently configurable. The defaults are sensible for most deployments; here is when you would change them:

Startup warnings for disabled bypasses

If you disable any high-risk bypass, the proxy emits a `WARN` log at startup and increments a Prometheus counter. This is intentional — it is a reminder that a non-default bypass configuration is active. The warning format:

```
WARN | policy | event=bypass_disabled | bypass=
    alpn_browser_bypass |
      effect=browser traffic will be scored; false
```

Bypass	When to disable
`alpn_browser_bypass`	Almost never. Disabling this means legitimate browsers can be blocked, which is the primary false-positive risk. Only disable if scoring specific h2 API clients is critical to your use case.
`ja4_whitelist_bypass`	If you want whitelisted fingerprints to be scored for observability purposes but still allowed.
`mtls_bypass`	If you want to score mTLS clients rather than unconditionally trust them.
`spamhaus_bypass`	If you want to investigate traffic from Spamhaus-listed ranges rather than hard-blocking it. Disabling routes those connections through the scorer with a +80 risk signal contribution.
`tls_version_bypass`	If you need to score rather than immediately reject TLS 1.0/1.1 clients — e.g., to quantify how much legacy TLS your infrastructure actually receives.

Table 7: When to disable bypass conditions

```
      positive risk elevated
```

Additional Configuration Sections

Signal Weights

Individual signal weights can be tuned in the `scorer` section. The defaults are calibrated from empirical testing; only adjust them if you have clear evidence that a specific signal is over- or under-contributing in your environment. *See the Technical Reference Manual for signal weight documentation.*

Beaconing Detection

```
beaconing:
  enabled: true
  min_connections: 5       # minimum connections before
    analysis
  cv_threshold: 0.3        # coefficient of variation threshold
  window_seconds: 3600     # look-back window
```

The beaconing detector looks for connections from the same IP+JA4 pair at suspiciously regular intervals. A coefficient of variation below the threshold (very regular timing) triggers a risk signal. The default threshold of 0.3 catches C2 beaconing without flagging APIs with regular polling intervals.

AbuseIPDB

```yaml
abuseipdb:
  enabled: true            # set to false if no API key
  api_key: ""              # or set via ABUSEIPDB_API_KEY env
    var
  score_threshold: 50      # minimum score to generate a risk
    signal
  cache_ttl_seconds: 3600  # cache successful lookups for 1
    hour
  hard_block_threshold: 0  # 0 = never hard-block via AbuseIPDB
     alone
```

AbuseIPDB score is used as one signal among many. The `hard_block_threshold` defaults to 0 (disabled) because AbuseIPDB scores reflect community reports that may include shared IPs (NAT, VPNs). A high AbuseIPDB score contributes to the composite risk score, but does not by itself produce a hard block.

Redis Connection

```yaml
redis:
  host: redis              # hostname or socket path
  port: 6379
  db: 0
  # unix_socket: /var/run/redis/redis.sock  # 30-40% lower
    latency than TCP
```

The Unix domain socket option is significantly faster than TCP for same-host Redis. If your proxy and Redis run on the same host, configure the socket path.

Day-to-Day Operations

Running JA4proxy day-to-day is mostly unremarkable. The proxy handles connections, Redis holds state, and Grafana shows you what is happening. This chapter covers the routine operational tasks: starting and stopping the stack, updating lists, managing bans, maintaining the GeoIP database, and keeping the proxy healthy.

Starting and Stopping

Normal Start

```
make start
```

Starts all containers. Existing Redis data (bans, rate-limit windows) is preserved. The proxy begins processing connections as soon as it becomes healthy, which takes approximately 5–10 seconds after the containers start.

Stop Without Data Loss

```
make stop
```

Stops all containers. Redis data persists on the host volume. Logs are preserved. Use this for planned maintenance or before deploying a configuration change that requires a restart.

Stop and Clean Everything

```
make stop-clean
```

Stops all containers and removes volumes. Use this when you want a completely fresh start — for example, when moving from testing to a fresh production deployment, or when debugging requires a known-clean Redis state.

Data loss

`make stop-clean` permanently deletes all Redis data: bans, rate-limit windows, beaconing history, enrichment cache, and return visitor records. Any active bans will be lifted immediately.

Full Rebuild

After pulling a new version of the repository:

```
git pull
make rebuild
```

This stops the stack, rebuilds all Docker images from scratch, and starts the new version.

Updating Fingerprint Lists

The JA4 blacklist, whitelist, patterns, and labels all live in `config/proxy.yml` under `security`. To add or remove entries:

1. Edit `config/proxy.yml`
2. Reload without restart:

```
kill -HUP $(docker compose exec ja4proxy cat /var/run/proxy.pid)
```

3. Verify the reload in logs:

```
docker compose logs ja4proxy | tail -5
# Should show: INFO | config | event=reload | status=ok
```

Adding a Fingerprint to the Blacklist

When you identify a malicious fingerprint (from logs, Grafana, or threat intelligence):

```
security:
  blacklist:
    - t13d0912_a1b2c3d4e5f6_987654321abc  # Sliver C2 - added 2026-04-04
    - t12d1302_c0ffee123456_aabbccddeeff  # Cobalt Strike - added 2026-04-04
    # Add your new entry here:
    - t13d1015_newfingerprint_0123456789  # Description - date
```

Comment each entry with a description and the date it was added. This is your operational audit trail.

Adding a Fingerprint to the Whitelist

For a legitimate tool that is scoring too high:

```
security:
  whitelist:
```

```
    - t13d1113h2_8daaf6152771_b0da82dd1658  # Chrome 120 (
    Windows)
    # Add your tool here:
    - t13d0914h2_abcdef123456_fedcba654321  # Internal
    monitoring agent
```

Whitelist changes are immediate

Whitelist additions take effect on reload and apply to all subsequent connections. They do not apply retroactively to active bans — if a whitelisted IP is currently banned, the ban persists until it expires (5 minutes).

Managing IP Bans

Viewing Current Bans

```
# List all active bans with TTLs
docker compose exec redis redis-cli --scan --pattern "ban:*" |
    \
  xargs -I{} sh -c 'echo -n "{}: "; docker compose exec redis
    redis-cli TTL "{}"'
```

Or for a quick count:

```
docker compose exec redis redis-cli --scan --pattern "ban:*" |
    wc -l
```

Clearing a Specific Ban

If a legitimate IP has been incorrectly banned:

```
# Replace with the actual IP address
docker compose exec redis redis-cli DEL "ban:192.0.2.1"
```

The next connection from that IP will be processed normally. There is no need to restart anything.

Ban Expiry

All bans have a 5-minute TTL.[8] You do not need to manually clear bans — they expire automatically. Manual clearing is only needed when you need to restore access immediately (e.g., during an incident where a legitimate user has been blocked).

[8] The self-healing TTL is intentional and cannot be disabled. See Section 1.5 for the rationale.

Clearing All Bans

Use this with caution — it immediately releases all currently-banned IPs:

```
docker compose exec redis redis-cli --scan --pattern "ban:*" |
    \
  xargs docker compose exec redis redis-cli DEL
```

Managing Rate-Limit Windows

Rate-limit sliding windows are stored as Redis Sorted Sets. To clear rate-limit state for a specific IP (for example, after a customer reports being rate-limited):

```
# Clear by-IP rate limit for a specific address
docker compose exec redis redis-cli DEL "rl:ip:192.0.2.1"

# Clear by-IP+JA4 pair rate limit
docker compose exec redis redis-cli DEL "rl:pair:192.0.2.1:
    t13d1113h2_8daaf6152771_b0da82dd1658"
```

To clear all rate-limit windows (useful between load tests):

```
docker compose exec redis redis-cli --scan --pattern "rl:*" | \
  xargs docker compose exec redis redis-cli DEL
```

Updating the GeoIP Database

The IP2Location database should be refreshed monthly. IP allocations change; a stale database will misclassify newer IP ranges.

```
make update-geoip
```

The proxy reloads the database on the next `SIGHUP` or can be made to reload it immediately:

```
# Reload everything including the GeoIP database
kill -HUP $(docker compose exec ja4proxy cat /var/run/proxy.pid
    )
```

If the database file is missing or corrupted, the proxy falls back to treating all IPs as country code `XX` (unknown). Country filtering is effectively disabled in this state, and the proxy logs a startup warning. Other functions are unaffected.

Flushing Redis Between Test Runs

When running load tests or switching between test scenarios, flush Redis to start with a clean state:

```
docker compose exec redis redis-cli FLUSHALL
```

FLUSHALL in production

Never run `FLUSHALL` on a production Redis instance. It removes all data including bans and enrichment caches. Use it only in development or test environments.

Checking Proxy Health

Quick Health Check

```
curl -s http://localhost:8080/health
```

Returns JSON with status, Redis connectivity, GeoIP status, and uptime.

Readiness Check

```
curl -s http://localhost:8080/ready
```

Returns 200 when the proxy is ready to accept connections. Used by Kubernetes readiness probes.

Container Status

```
docker compose ps
docker compose top
```

Resource Usage

```
docker stats --no-stream
```

Shows CPU, memory, and network I/O for all containers. The proxy container should be using well under 100% CPU at normal load; spikes above this indicate a performance issue.

Log Management

Logs are collected by Promtail and stored in Loki. The Grafana Explore interface queries them directly. For direct log access:

```
# Live proxy logs
docker compose logs -f ja4proxy

# Last 100 lines of all services
docker compose logs --tail=100

# Logs since a specific time
docker compose logs --since=2026-04-04T10:00:00 ja4proxy
```

Log files are also written to `logs/` on the host. Default retention is 7 days; adjust in `config/proxy.yml` under `logging.retention_days`.

Scheduled Maintenance Checklist

Frequency	Task	Command
Daily	Review Grafana dashboard	Visual check: action distribution, score histogram
Daily	Check for new fingerprints	Grafana Top Fingerprints panel
Weekly	Review ban log	Check for patterns suggesting targeting
Monthly	Update GeoIP database	`make update-geoip`
Monthly	Review blacklist	Remove stale entries, add newly identified tools
Monthly	Check Docker image updates	`git pull && make rebuild`

Table 8: Recommended maintenance schedule

Monitoring and Alerting

The observability stack is Prometheus, Grafana, Loki, and Alertmanager. They run in the same Compose stack as the proxy and require no external setup. This chapter explains what each panel in the Grafana dashboard means, which Prometheus metrics matter most, how to query logs in Loki, and what alerts are pre-configured.

The Observability Stack

Service	URL	Purpose
Grafana	`http://localhost:3001`	Dashboard, log exploration, alerting UI
Prometheus	`http://localhost:9091`	Metrics storage and query (PromQL)
Loki	`http://localhost:3100`	Log aggregation (queried via Grafana)
Alertmanager	`http://localhost:9093`	Alert routing and silencing

Table 9: Observability service endpoints

The proxy exposes its Prometheus metrics endpoint at `http://localhost:9090`. Prometheus scrapes this every 15 seconds.[9]

[9] If you run multiple proxy instances, each instance has its own metrics endpoint. Prometheus is pre-configured to scrape all of them via service discovery.

The Grafana Dashboard

Open `http://localhost:3001` and select the *JA4proxy Overview* dashboard. The default time range is the last hour; use the time picker in the top right to adjust.

Connection Rate Panel

The top-line panel. Shows connections per second, stacked by action type.

What to look for:

- A sudden spike in the red (BLOCKED) area indicates an active attack has been detected and is being blocked.
- A spike in total connections without a corresponding increase in BLOCKED connections may indicate a new attack type that the proxy is not yet classifying correctly — or that the dial is at zero.

- A sudden drop to zero in all lines means the proxy has stopped accepting connections. Check the container status.

Action Distribution Panel

Shows the percentage breakdown of actions over the selected time window. In a well-tuned deployment with the dial at 100, typical values are:

- 60–80% ALLOWED (legitimate traffic)
- 15–35% BLOCKED (confirmed bad traffic)
- 1–5% TARPITTED (high-risk, being slowed)
- <1% RATE_LIMITED (automated clients hitting limits)
- 0% at dial=0 (everything is ALLOWED)

If BLOCKED is above 50%, either your traffic is heavily automated or a bypass condition needs tuning. Investigate before raising the dial further.

Risk Score Distribution Panel

A histogram of risk scores (0–100) across all scored connections. The shape tells you about the quality of your traffic classification:

Bimodal Peaks near 0–10 and 80–100. Ideal. Legitimate traffic is scoring low, bad traffic is scoring high, with a clear separation.

Left-skewed Most traffic near 0, long tail to the right. Typical for low-attack-volume deployments. The tail represents the bad traffic.

Flat/uniform Signals may be under-tuned or traffic is very mixed. Review which signals are contributing most.

Right-skewed Most traffic scoring high. Either the proxy is seeing a lot of automated traffic, or whitelist entries are needed for legitimate tools.

Top JA4 Fingerprints Panel

A ranked table of fingerprints seen in the selected window, with:

- Connection count
- Average risk score
- Human-readable label (if configured)
- Action distribution for that fingerprint

Workflow: Look for high-volume fingerprints with high average scores that are currently being ALLOWED (dial=0 or dial too low). These are your blacklist candidates. Look for high-volume fingerprints with high average scores that are BLOCKED — verify these are genuinely malicious before keeping them blocked.

Geographic Distribution Panel

Connection count by country. Useful for:

- Verifying your GeoIP database is working (you should see your own country)
- Detecting a volumetric attack from a specific region
- Validating country whitelist/blacklist configuration

Beaconing Activity Panel

Shows IPs currently flagged by the beaconing detector. Regular-interval connections from the same IP+JA4 pair suggest C2 traffic. The panel shows the IAT coefficient of variation — values below the threshold indicate suspicious regularity.

Rate Limiting Panel

Breakdown of rate-limit triggers by strategy (by_ip, by_ja4, by_ip_ja4_pair). High sustained activity in by_ip suggests volumetric attacks from single sources. High activity in by_ja4 suggests a distributed botnet using consistent tooling.

Key Prometheus Metrics

Connection Metrics

```
# Total connections by action
ja4proxy_connections_total{action="allowed|blocked|tarpitted|
    rate_limited"}

# Active connections right now
ja4proxy_active_connections

# Connection processing duration
ja4proxy_connection_duration_seconds{quantile="0.5|0.95|0.99"}
```

Risk Scoring

```
# Score distribution histogram
ja4proxy_risk_score_distribution{le="10|25|50|75|100"}

# Current dial setting
ja4proxy_dial_setting
```

Signal Performance

```
# Per-signal hit rates
ja4proxy_signal_hits_total{signal="abuseipdb|beaconing|asn|
    tls_version|..."}
```

```
# External service latency (AbuseIPDB, RDAP)
ja4proxy_external_lookup_duration_seconds{service="abuseipdb|
    rdap"}

# External service errors (should be near zero)
ja4proxy_external_lookup_errors_total{service="abuseipdb|rdap"}
```

Redis Performance

```
# Redis operation latency
ja4proxy_redis_operation_duration_seconds{operation="get|set|
    zadd|..."}

# Redis errors (critical - should be zero in normal operation)
ja4proxy_redis_errors_total{operation="..."}
```

Useful PromQL Queries

Block rate over the last 5 minutes:

```
rate(ja4proxy_connections_total{action="blocked"}[5m]) /
  rate(ja4proxy_connections_total[5m])
```

Average risk score (approximated from histogram):

```
histogram_quantile(0.50,
  rate(ja4proxy_risk_score_distribution_bucket[5m]))
```

AbuseIPDB error rate (should alert if non-zero):

```
rate(ja4proxy_external_lookup_errors_total{service="abuseipdb"
    }[5m])
```

Loki Log Queries

Access Loki through the Grafana Explore interface: navigate to Explore and select the Loki data source.

Common LogQL Queries

Find all blocked connections in the last hour:

```
{container="ja4proxy"} |= "BLOCKED:"
```

Find all connections from a specific IP:

```
{container="ja4proxy"} |= "185.220.0.1"
```

Find all connections with a specific fingerprint:

```
{container="ja4proxy"} |= "t13d0912_a1b2c3d4e5f6"
```

Find high-scoring connections (score above 70):

```
{container="ja4proxy"} |= "Score:" | regexp "Score: (?P<score
    >[789][0-9]|100)"
```

Count block events by country over time:

```
sum by (country) (
  count_over_time(
    {container="ja4proxy"} |= "BLOCKED:" | regexp "Country: (?P
    <country>[A-Z]{2})"
    [5m]
  )
)
```

Pre-Configured Alerts

The following alerts are pre-configured in Alertmanager. Review and adjust thresholds to match your environment before deploying to production.

Table 10: Pre-configured alert rules

Alert	Condition	Severity
`ProxyDown`	Proxy metrics endpoint unreachable for 1m	Critical
`RedisDown`	Redis health check failing for 1m	Critical
`HighBlockRate`	Block rate >80% for 5m	Warning
`HighRiskScores`	P95 risk score >75 for 10m	Warning
`AbuseIPDBErrors`	AbuseIPDB error rate >0 for 5m	Info
`ScoreDrift`	Mean risk score drifts >20 points from 24h baseline	Warning
`BeaconingSpike`	Beaconing detections > 10× baseline for 5m	Warning
`RedisLatency`	P99 Redis operation >5ms for 5m	Warning

Score drift alerting

The `ScoreDrift` alert is particularly useful. A sudden increase in mean risk scores across all connections often indicates a new attack campaign has started. A sudden decrease may indicate a bypass condition was incorrectly configured or that attackers have adapted their fingerprints.

Configuring Alert Destinations

Alert destinations (email, PagerDuty, Slack) are configured in `config/alertmanager.yml`. The file ships with a commented-out template

for each integration type. *See the Alertmanager documentation for receiver configuration.*

Accessing Logs Directly

For quick investigations without the Grafana interface:

```
# All proxy logs, live
docker compose logs -f ja4proxy

# Last 500 lines, grep for errors
docker compose logs --tail=500 ja4proxy | grep -E "ERROR|WARN"

# Analytics node logs (campaign detection, slow-scan)
docker compose logs -f analytics

# HAProxy access log (raw TCP connections)
docker compose logs -f haproxy
```

Incident Response

An active attack looks different depending on its type. A fingerprint-consistent botnet shows up immediately in the Grafana fingerprint panel. A slow scan with randomised tool configurations might only show up in the analytics node's campaign detection. A sudden volumetric spike shows up in the connection rate panel within seconds.

This chapter gives you a structured approach to detecting, responding to, and recovering from incidents involving JA4proxy.

Detecting an Active Attack

What to Look For in Grafana

Open the *JA4proxy Overview* dashboard and look for these indicators:

Connection rate spike A sudden increase in connections/second, especially if the action distribution shows a high BLOCKED fraction. This is a good sign — the proxy is detecting and handling the attack.

Risk score distribution shift The histogram shifts right (higher scores). This indicates more traffic scoring as suspicious.

New dominant fingerprint A single fingerprint appears at the top of the Top Fingerprints table with high connection counts and high average scores.

Geographic concentration The map shows a sudden spike from a region you do not normally serve.

Beaconing spikes The analytics node detects a cluster of IPs beaconing at similar intervals — characteristic of C2 callback traffic.

Using the Score Drift Alert

The `ScoreDrift` Alertmanager alert fires when the mean risk score across all connections drifts more than 20 points from the 24-hour

baseline. This catches attacks that do not produce obvious connection rate spikes — such as slow scanners or distributed credential-stuffing campaigns spread across many source IPs.

Checking the Analytics Node

The analytics node performs cross-instance aggregation and campaign detection. Its findings are pushed to Redis and read by the scorer. Check its logs for detected patterns:

```
docker compose logs analytics | grep -E "campaign|slow.scan|
    score.drift"
```

Emergency Blocking Procedures

Blocking a JA4 Fingerprint Immediately

When you identify a malicious fingerprint and need to block it immediately:

1. Add it to the blacklist in `config/proxy.yml`:

```
security:
  blacklist:
    - t13d0912_a1b2c3d4e5f6_987654321abc  # Sliver C2 -
      2026-04-04 incident
```

2. Reload config:

```
kill -HUP $(docker compose exec ja4proxy cat /var/run/proxy.
    pid)
```

3. Verify in logs that subsequent connections from that fingerprint are blocked:

```
docker compose logs -f ja4proxy | grep "JA4 blacklisted"
```

Blocking via Redis directly

For even faster response (before you can edit the config file), you can add a fingerprint to the Redis blacklist set directly:

```
docker compose exec redis redis-cli SADD ja4:blacklist \
  "t13d0912_a1b2c3d4e5f6_987654321abc"
```

This takes effect immediately for new connections. Remember to also add it to `config/proxy.yml` to persist the change through restarts.

Blocking a Specific IP Address

To immediately ban an IP for the standard 5-minute TTL:

```
docker compose exec redis redis-cli SETEX "ban:185.220.0.1" 300
      "manual"
```

For a longer ban (e.g., 1 hour = 3600 seconds):

```
docker compose exec redis redis-cli SETEX "ban:185.220.0.1"
     3600 "manual-1h"
```

Ban TTLs and self-healing

There are no permanent bans in JA4proxy. The maximum practical ban duration is whatever you set manually in Redis — but be careful with long ban durations. Shared IP infrastructure (NAT, corporate proxies, CDN exit nodes) means long bans can affect many legitimate users sharing an IP.

Blocking a CIDR Range

JA4proxy does not currently support native CIDR banning via Redis (CIDR matching is in-process via a trie, not Redis). For immediate CIDR blocking, use HAProxy's ACL mechanism or your network firewall.

For example, to block `185.220.0.0/24` at the HAProxy level:

```
# Add to haproxy.cfg in the frontend section:
acl blocked_range src 185.220.0.0/24
tcp-request connection reject if blocked_range
```

Then reload HAProxy:

```
docker compose kill -s HUP haproxy
```

Emergency Country Block

To block an entire country immediately via config:

```
country_blacklist_enabled: true
country_blacklist:
  - RU   # Added 2026-04-04 - active attack campaign
```

Reload config. The block takes effect for new connections immediately.

Investigating a Suspicious Fingerprint

Step 1: Identify the Fingerprint

From Grafana's Top Fingerprints panel, or from logs:

```
docker compose logs ja4proxy | grep "Score: [7-9][0-9]\|Score:
    100" | \
  grep -oP "JA4: \K[^ |]+" | sort | uniq -c | sort -rn | head
    -20
```

Step 2: Look It Up

Search the JA4 community database: https://ja4db.com

The database contains community-identified labels for thousands of fingerprints, including known C2 frameworks, scanners, and security tools.

Step 3: Query Redis for Activity

Check the rate-limit history for this fingerprint:

```
docker compose exec redis redis-cli \
  ZCARD "rl:ja4:t13d0912_a1b2c3d4e5f6_987654321abc"
```

Check beaconing records:

```
docker compose exec redis redis-cli --scan \
  --pattern "beacon:*:t13d0912_a1b2c3d4e5f6_987654321abc"
```

Step 4: Assess Risk

Ask:

- Is this fingerprint in the JA4 database as a known malicious tool?
- What is its average risk score? (Grafana fingerprint panel)
- How many source IPs are using it? (Redis PFCOUNT on the HLL key)
- Is it beaconing at regular intervals?
- Is it associated with known-bad ASNs or countries?

If the answer to the first question is yes and the fingerprint is not shared with any legitimate tool, add it to the blacklist. If you are unsure, add it to the watchlist and continue monitoring for 24–48 hours.

Escalating to a Permanent Country Block

Country blocking is a significant decision — it will affect all users from that country, including legitimate ones. Follow this procedure:

1. Verify the attack is predominantly from the target country (Grafana geographic distribution panel).
2. Check how much legitimate traffic you receive from that country (filter Grafana for the country code and check the action distribution).
3. If legitimate traffic from that country is $<1\%$ of total, the block has acceptable false-positive impact.
4. Add to the blacklist in `config/proxy.yml`. Note: if `country_whitelist_enabled` is true, add the excluded countries to the whitelist instead.
5. Reload config and monitor the block rate for 30 minutes.
6. Document the decision: which country, why, what the evidence was, who approved it.

Recovering From a False Positive

A false positive occurs when a legitimate connection is blocked or banned. The most common causes are:

- A legitimate tool uses a fingerprint that also appears in the blacklist
- A legitimate IP has been auto-banned due to high request volume
- A country block is affecting legitimate users

Immediate Recovery

If a user is currently blocked:

```
# Clear their ban immediately
docker compose exec redis redis-cli DEL "ban:192.0.2.1"
```

If a fingerprint was incorrectly blacklisted:

1. Remove the fingerprint from `security.blacklist` in `config/proxy.yml`
2. Reload config
3. If the fingerprint was also added to the Redis blacklist directly, remove it there too:

```
docker compose exec redis redis-cli SREM ja4:blacklist \
  "t13d1113h2_8daaf6152771_b0da82dd1658"
```

Root Cause and Prevention

After clearing the false positive:

1. Identify what triggered it (which bypass condition or scoring signal)

2. If a specific fingerprint was too broadly blacklisted, consider whether the problematic tool shares a fingerprint with legitimate software
3. If the rate limit was too aggressive, adjust the threshold in `config/proxy.yml` and reload
4. Add the legitimate fingerprint to the whitelist to prevent recurrence
5. Document the incident and the configuration change

Post-Incident Review

After resolving any significant incident:

1. Export Grafana panel data for the incident time window (use the panel's share/export function)
2. Pull the log segment from Loki covering the incident
3. Record:
   - When it started and when it was resolved
   - The attack type (fingerprint-based, volumetric, geographic)
   - Which signals detected it first
   - What actions were taken
   - Whether any false positives occurred
4. Update the fingerprint blacklist, labels, or rate-limit thresholds based on what you learned
5. Consider whether a new alert rule would have caught this faster

Preserving incident data

Grafana's snapshot feature captures the current state of the dashboard for a specific time range. Save a snapshot for significant incidents before the data rolls out of the retention window:

```
Dashboard menu -> Share -> Snapshot -> Publish
```

Scaling

Scaling JA4proxy is straightforward because the proxy is stateless per-instance. All shared state (bans, blacklists, rate-limit windows, enrichment caches) lives in Redis, which every proxy instance reads and writes. Adding more proxy instances means adding more connection processing capacity without any coordination overhead between instances.

This chapter covers horizontal scaling with Docker Compose, the Kubernetes deployment option, and guidance on when to consider the Go proxy.

Performance Baseline

Before scaling, it helps to understand the single-instance baseline:

Configuration	Throughput
Single Python instance, Redis pipeline batching	~350 connections/second
Single Python instance, in-process cache only	~550 connections/second
4 Python instances, Redis	~1400 connections/second
Go proxy (single instance, experimental)	Targeting 3500–17 500+ conn/s

Table 11: Throughput benchmarks by configuration

The bottleneck at high load is Redis round-trip time for signal collection writes. Pipeline batching in the proxy reduces this overhead significantly; the Unix socket connection to Redis reduces it further.

Docker Compose Scaling

Scaling to Multiple Instances

```
./scripts/scale-proxies.sh 4
```

This adjusts the Docker Compose configuration to run 4 proxy instances and reloads HAProxy's backend configuration to load-balance

across them. You should see all 4 instances appear in the HAProxy statistics page at http://localhost:8404/stats.

HAProxy uses round-robin load balancing across proxy instances. Each proxy instance registers its own metrics endpoint; Prometheus is pre-configured to scrape all of them via the Docker service discovery mechanism.

What Is and Is Not Shared

Table 12: State sharing between proxy instances

State	Where stored	Shared across instances?
IP bans	Redis (ban:* keys)	Yes — all instances see all bans
JA4 blacklist	Redis SET + in-process	Yes — synced from Redis at startup and reload
JA4 whitelist	Redis SET + in-process	Yes
Rate-limit windows	Redis Sorted Sets	Yes — truly cluster-wide sliding windows
Beaconing history	Redis Sorted Sets	Yes
GeoIP database	Read-only mmap file	No — each instance reads its own copy
Local decision cache	In-process LRU	No — per-instance, provides fast-path for repeat visitors
Enrichment cache	Redis	Yes
Analytics findings	Redis	Yes

The local in-process cache is per-instance and not shared. This means a client whose connection was allow-cached on instance A may be re-scored on instance B. This is acceptable: the cache is an optimisation, not a correctness requirement. The first-time score is the authoritative result; the cache prevents re-scoring the same fingerprint+IP combination on the same instance.

HAProxy Configuration

HAProxy's configuration is in `config/haproxy.cfg`. For the development stack, it is auto-generated by `scripts/scale-proxies.sh`. For production deployments, you will manage this directly.

The key section for proxy backend configuration:

```
backend ja4proxy_backend
    balance roundrobin
    option tcp-check
    server ja4proxy-1 ja4proxy-1:8080 check inter 2s fall 3
     rise 2
    server ja4proxy-2 ja4proxy-2:8080 check inter 2s fall 3
     rise 2
    server ja4proxy-3 ja4proxy-3:8080 check inter 2s fall 3
     rise 2
    server ja4proxy-4 ja4proxy-4:8080 check inter 2s fall 3
     rise 2
```

HAProxy performs TCP health checks every 2 seconds. If 3 consecutive checks fail, the instance is removed from rotation. When 2 consecutive checks pass, it is returned to rotation. This provides fast failure detection without flapping.

Redis Scaling Considerations

A single Redis instance handles the shared state for all proxy instances. At 4 proxy instances with Redis pipeline batching, Redis is typically below 10% CPU utilisation. At 8+ instances or very high connection rates, consider:

- Enabling the Unix domain socket connection (reduces per-operation overhead)
- Increasing Redis's `maxmemory` and enabling an appropriate eviction policy for enrichment caches
- For production deployments: Redis Sentinel for HA, or Redis Cluster for horizontal scaling

Redis is a single point of failure

In the development stack, a single Redis instance handles all shared state. If Redis becomes unavailable, the proxy fails open (allows all connections) rather than failing closed (blocking everything). This is the correct behaviour for a security proxy, but it means your defence is weakened during Redis downtime. For production: deploy Redis with Sentinel (3 nodes) for automatic failover.

Kubernetes Deployment

JA4proxy ships with a Helm chart in `deploy/helm/ja4proxy/`.

Basic Installation

```
helm install ja4proxy ./deploy/helm/ja4proxy \
```

```
  --set proxy.backendHost=prod.internal.example.com \
  --namespace ja4proxy \
  --create-namespace
```

Key Helm Values

Table 13: Key Helm chart values

Value	Default	Description
`proxy.backendHost`	(required)	Hostname of backend server
`proxy.dialSetting`	`0`	Initial dial value (0 = monitor)
`proxy.replicas`	`2`	Initial replica count
`hpa.enabled`	`true`	Enable horizontal pod autoscaler
`hpa.minReplicas`	`2`	Minimum pod count
`hpa.maxReplicas`	`20`	Maximum pod count
`hpa.targetCPU`	`70`	CPU utilisation target (%)
`redis.external`	`true`	Use external Redis (recommended)
`redis.host`	`redis`	Redis hostname
`redis.port`	`6379`	Redis port
`resources.requests.cpu`	`500m`	CPU request per pod
`resources.requests.memory`	`256Mi`	Memory request per pod
`resources.limits.cpu`	`2000m`	CPU limit per pod
`resources.limits.memory`	`512Mi`	Memory limit per pod

Production Kubernetes Setup

For a production Kubernetes deployment:

```
helm install ja4proxy ./deploy/helm/ja4proxy \
  --set proxy.backendHost=prod.internal.example.com \
  --set proxy.dialSetting=0 \
  --set proxy.replicas=3 \
  --set hpa.maxReplicas=20 \
  --set redis.external=true \
  --set redis.host=redis.prod-infra.svc.cluster.local \
  --set resources.requests.cpu=500m \
  --set resources.limits.cpu=2000m \
  --namespace ja4proxy \
  --create-namespace
```

External Redis for Kubernetes

Set `redis.external=true` and provide your own Redis instance (Redis Sentinel or Redis Cluster) for Kubernetes deployments. The bundled Redis in the Helm chart is for development only — it has no persistence or HA configuration.

Horizontal Pod Autoscaler

The HPA is configured to scale on CPU utilisation. At 70% CPU across the pod set, Kubernetes adds pods up to `hpa.maxReplicas`. The

scale-down cooldown is 5 minutes to prevent thrashing.

Monitor HPA status:

```
kubectl get hpa -n ja4proxy
kubectl describe hpa ja4proxy -n ja4proxy
```

Configuration in Kubernetes

The `config/proxy.yml` content is mounted as a ConfigMap. To update configuration in Kubernetes:

```
# Update the ConfigMap
kubectl create configmap ja4proxy-config \
  --from-file=proxy.yml=config/proxy.yml \
  --namespace ja4proxy \
  --dry-run=client -o yaml | kubectl apply -f -

# Trigger a rolling reload by annotating the deployment
kubectl rollout restart deployment/ja4proxy -n ja4proxy
```

Hot reload in Kubernetes

Hot reload via SIGHUP works in Kubernetes if you exec into the pod. For a cluster-wide config reload (all pods), it is simpler to do a rolling restart as shown above. A rolling restart replaces pods one at a time, maintaining availability.

When to Consider the Go Proxy

The Python proxy is the stable implementation. Consider the Go proxy when:

- Your sustained connection rate exceeds 1200 connections/second with 4 Python instances (at that point, you are adding too many instances)
- You need sub-millisecond processing latency
- You are deploying on hardware-constrained infrastructure where the Python interpreter overhead is significant
- You have capacity to validate and monitor a newer implementation

The Go proxy is not a simple drop-in replacement. It requires Go 1.22+ and a separate build step, and some auxiliary integrations (analytics, management UI) are still being finalised. See the current parity status in `docs/phases/PHASE_15.md` before using it in production.

Troubleshooting

Most JA4proxy problems fall into one of five categories: containers that will not start, traffic not being blocked when it should be, traffic being blocked when it should not be, performance problems, and connectivity issues between components. This chapter covers all five.

Start with `docker compose ps` and `docker compose logs` for any problem — most issues produce clear error messages in the container logs.

Quick Diagnostic Commands

These commands give you a fast overview of what is happening:

```
# Container status and health
docker compose ps

# All container logs, last 50 lines each
docker compose logs --tail=50

# Proxy-specific logs, live
docker compose logs -f ja4proxy

# Redis connectivity test
docker compose exec redis redis-cli PING

# Proxy health endpoint
curl -s http://localhost:8080/health | python3 -m json.tool

# HAProxy stats
curl -s http://localhost:8404/stats

# Active ban count
docker compose exec redis redis-cli --scan --pattern "ban:*" |
    wc -l

# Current dial setting
docker compose exec redis redis-cli GET "config:dial"
```

Symptom Lookup Table

Container Will Not Start

Table 14: Container startup failures

Symptom	Likely cause	Fix
`Error: port 443 in use`	Another process is on port 443	Stop the conflicting service or change the port mapping in `docker-compose.yml`
`redis: connection refused`	Redis did not start before the proxy	Wait 10s and retry; Redis has a health check dependency. Run `docker compose restart ja4proxy`
`No such file: proxy.yml`	Config file missing from expected path	Ensure `config/proxy.yml` exists. It is not auto-created; restore from `config/proxy.yml.example`
`YAML parse error`	Syntax error in `config/proxy.yml`	Validate with `python3 -c "import yaml; yaml.safe_load(open('config/proxy.yml'))"`
Build fails at pip install	Network issue or PyPI unavailable	Try `docker compose build --no-cache ja4proxy` with a working network connection

Proxy Running but Not Blocking

Legitimate Traffic Being Blocked

High Latency

Redis Connection Errors

Metrics Not Appearing in Grafana

Log Analysis for Common Issues

Finding the Root Cause of a Block

When a connection is blocked and you want to know why:

```
# Search for a specific IP in logs
docker compose logs ja4proxy | grep "192.0.2.1"
```

Table 15: Traffic not being blocked as expected

Symptom	Likely cause	Fix
All traffic shows ALLOWED	Dial is 0 (monitor mode)	This is correct behaviour. Raise the dial in `config/proxy.yml` when ready
Known-bad fingerprint not blocked	JA4 blacklist bypass disabled in config	Check `security_policy.ja4_blacklist_bypass.enabled`
Blacklisted fingerprint not blocked	Fingerprint not in Redis SET	After config reload, verify: `redis-cli SISMEMBER ja4:blacklist "fingerprint"`
Country block not working	GeoIP database missing or stale	Run `make update-geoip` and reload config
High-score traffic not blocked	Dial too low for score	Check the score in logs and compare to the block threshold at your current dial setting

The `Reason:` field in the log line identifies the blocking cause:

Checking Signal Contributions

At `DEBUG` log level, the proxy logs individual signal contributions. Enable debug logging temporarily:

```
logging:
  level: DEBUG   # Change from INFO to DEBUG
```

Reload config. Debug output includes each signal's score contribution:

```
DEBUG | scorer | ip=185.220.0.1 | signal=abuseipdb | contrib
    =+30 | reason=score=75
DEBUG | scorer | ip=185.220.0.1 | signal=asn | contrib=+20 |
    reason=datacenter_asn
DEBUG | scorer | ip=185.220.0.1 | signal=beaconing | contrib
    =+40 | reason=cv=0.12
DEBUG | scorer | ip=185.220.0.1 | total_score=90
```

Table 16: False positives — legitimate traffic blocked

Symptom	Likely cause	Fix
All browser traffic blocked	ALPN bypass disabled	Set `security_policy.alpn_browser_bypass.enabled: true` and reload
Specific legitimate tool blocked	Tool fingerprint blacklisted	Move fingerprint from blacklist to whitelist, or remove from blacklist
API client rate-limited	Rate limit too aggressive for API use case	Add client fingerprint to whitelist or increase rate limit thresholds
Customer reports blocked	IP in wrong GeoIP country	Check the country shown in the log entry; update GeoIP if stale
Monitoring tool blocked	Internal IP not in allowlist	Add monitoring agent's IP to `security.static_ip_allowlist`

Debug logging volume

At DEBUG level, every connection produces many log lines. On a busy proxy this generates significant log volume. Enable debug logging only for targeted investigations and disable it immediately after.

Getting Help

GitHub Issues `https://github.com/seanpor/JA4proxy/issues` — for bug reports and feature requests. Include the output of `docker compose logs` and the relevant section of `config/proxy.yml` (with any credentials redacted).

JA4 Database `https://ja4db.com` — for identifying unknown fingerprints.

JA4 Specification `https://github.com/FoxIO-LLC/ja4` — for understanding how fingerprints are computed.

Reference Manual The companion document for architecture detail, signal algorithm specifications, and API documentation.

When reporting an issue, include:

- The symptom (what you expected vs what happened)
- The relevant section of `config/proxy.yml`

- Output of `docker compose ps` and `docker compose logs --tail=100`
- The JA4proxy version (`git rev-parse --short HEAD`)
- The host OS and Docker version

Table 17: Latency problems

Symptom	Likely cause	Fix
P95 connection latency >5ms	Redis TCP round-trips	Switch to Unix domain socket: set `redis.unix_socket` in config
P99 latency spikes	AbuseIPDB API timeout	AbuseIPDB calls are fire-and-forget; they should not affect latency. Check if the AbuseIPDB timeout is set correctly
High latency on first connection from new IP	DNS lookup blocking	DNS enrichment uses async non-blocking lookups; verify `dns_enrichment.async: true`
Redis latency >2ms	Redis container resource-starved	Check `docker stats` for Redis CPU and memory; increase Docker memory limit

Symptom	Likely cause	Fix
Proxy logs: `Redis connection lost`	Redis container restarted	Proxy reconnects automatically; verify Redis is healthy
`READONLY error`	Redis has entered read-only mode	Check Redis logs; may indicate disk-full condition
`OOM command not allowed`	Redis hit `maxmemory` limit	Increase `maxmemory` in `config/redis.conf` or clear expired keys
All metrics show zero bans	Redis data was flushed	Check if something ran `FLUSHALL`; this is the expected state after `make stop-clean`

Table 18: Redis connectivity and data issues

Symptom	Likely cause	Fix
Grafana shows "No data"	Prometheus target is down	Check `http://localhost:9091/targets` — the ja4proxy target should be UP
Prometheus target UP but no data	No traffic has been processed	Generate test traffic with `generate-tls-traffic.sh`
Dashboard panels show different time ranges	Panel time range overrides set	Click each panel, edit, and remove time range overrides
Loki shows no logs	Promtail not running	Check `docker compose ps` for the promtail service

Table 19: Observability issues

Reason string	Cause
`JA4 blacklisted`	Fingerprint is in the blacklist
`Banned for 300s`	IP was auto-banned by rate limiter
`Country blocked: RU`	Country is in the blacklist
`TLS version rejected`	TLS 1.0 or 1.1
`Spamhaus DROP match`	IP is in Spamhaus DROP/EDROP list
`Score: 85 exceeds threshold`	Risk score above block threshold at current dial
`Rate limited by IP`	by_ip strategy triggered
`Rate limited by JA4`	by_ja4 strategy triggered

Table 20: Block reason strings and their causes

Quick Reference

Make Targets

Target	Effect
`make start`	Build images (if needed) and start the full stack
`make stop`	Stop all containers; preserve Redis data
`make stop-clean`	Stop and remove all containers and volumes
`make rebuild`	Wipe images and rebuild from scratch; wipes data
`make update-geoip`	Download and install IP2Location database
`make test`	Run full test suite
`make test-unit`	Unit tests only
`make logs`	Tail logs for all services

Docker Compose Commands

Command	Effect
`docker compose ps`	Show container status and health
`docker compose logs -f ja4proxy`	Live proxy logs
`docker compose logs -tail=100`	Last 100 lines, all services
`docker compose exec ja4proxy bash`	Shell into proxy container
`docker compose exec redis redis-cli`	Redis CLI session
`docker compose restart ja4proxy`	Restart proxy (preserves Redis)
`docker compose kill -s HUP haproxy`	Reload HAProxy config
`docker stats -no-stream`	Resource usage snapshot

Redis CLI — Common Operations

Prefix all commands with `docker compose exec redis` when running from the host:

```
docker compose exec redis redis-cli DEL "ban:1.2.3.4"
```

Config Reload

```
# Hot reload (no restart required)
kill -HUP $(docker compose exec ja4proxy cat /var/run/proxy.pid
    )
```

Command	Effect
`redis-cli PING`	Connectivity check
`redis-cli GET "ban:1.2.3.4"`	Check if IP is banned
`redis-cli DEL "ban:1.2.3.4"`	Clear a specific ban
`redis-cli SETEX "ban:1.2.3.4" 300 "manual"`	Ban IP for 5 minutes
`redis-cli -scan -pattern "ban:*" | wc -l`	Count active bans
`redis-cli SADD ja4:blacklist "fingerprint"`	Add to JA4 blacklist
`redis-cli SREM ja4:blacklist "fingerprint"`	Remove from blacklist
`redis-cli SISMEMBER ja4:blacklist "fp"`	Check if fingerprint is blacklisted
`redis-cli GET "config:dial"`	Read current dial value
`redis-cli SET "config:dial" "50"`	Set dial to 50 (affects new connections)
`redis-cli FLUSHALL`	**DEV ONLY** — wipe all Redis data

```
3
4  # Verify reload succeeded
5  docker compose logs ja4proxy | tail -3
6  # Expected: INFO | config | event=reload | status=ok
```

Traffic Generator

```
1  # Usage: ./scripts/generate-tls-traffic.sh <seconds> <legit_%>
       <workers>
2  ./scripts/generate-tls-traffic.sh 60 50 10   # 60s, 50% legit,
       10 workers
3  ./scripts/generate-tls-traffic.sh 30 10 20   # 30s, 10% legit,
       20 workers (stress test)
4  ./scripts/generate-tls-traffic.sh 300 80 5   # 5m, 80% legit (
       baseline capture)
```

Scaling

```
1  # Docker Compose scaling
2  ./scripts/scale-proxies.sh 4   # Scale to 4 proxy instances
3
4  # Kubernetes
5  helm install ja4proxy ./deploy/helm/ja4proxy \
6    --set proxy.backendHost=your.backend \
7    --namespace ja4proxy --create-namespace
8
9  kubectl get hpa -n ja4proxy    # Check autoscaler status
10 kubectl rollout restart deployment/ja4proxy -n ja4proxy  #
       Rolling restart
```

Port	Service	URL
443	HAProxy (TLS entry)	`https://localhost:443`
8080	JA4proxy	`http://localhost:8080`
8404	HAProxy stats	`http://localhost:8404/stats`
8443	Test backend	`https://localhost:8443`
8888	Tarpit	`http://localhost:8888`
9090	Proxy metrics	`http://localhost:9090/metrics`
9091	Prometheus	`http://localhost:9091`
3001	Grafana	`http://localhost:3001`
3100	Loki	`http://localhost:3100`
9093	Alertmanager	`http://localhost:9093`

Port Reference

Metric	What it measures
`ja4proxy_connections_total{action}`	Connections by action (allowed/blocked/tarpitted)
`ja4proxy_active_connections`	Live connection count right now
`ja4proxy_connection_duration_seconds`	Processing latency histogram
`ja4proxy_risk_score_distribution`	Score histogram (0–100)
`ja4proxy_dial_setting`	Current dial value
`ja4proxy_redis_errors_total`	Redis errors (alert if nonzero)
`ja4proxy_external_lookup_errors_total`	AbuseIPDB / RDAP errors
`ja4proxy_signal_hits_total{signal}`	Per-signal trigger counts

Key Prometheus Metrics

File	Purpose
`config/proxy.yml`	All proxy configuration (hot-reloadable)
`.env`	Environment variables (credentials, backend host)
`config/haproxy.cfg`	HAProxy load balancer configuration
`config/proxy.yml.example`	Config template with all keys documented
`logs/`	Proxy log files (host-mounted volume)
`data/geoip/`	IP2Location database files
`deploy/helm/ja4proxy/`	Kubernetes Helm chart
`scripts/scale-proxies.sh`	Horizontal scaling helper
`scripts/generate-tls-traffic.sh`	Test traffic generator

File Locations

Dial Quick Reference

Dial value	Behaviour	Use when
0	Monitor only — all traffic passes	Initial deployment; building baseline
25	Flag high-risk traffic	First week; reviewing score distribution
50	Rate limiting active	Confident in rate-limit thresholds
75	Tarpit + rate limit	Ready to slow down high-risk traffic
100	Full blocking	Production, after validating at lower settings

Index

JA4proxy

Enterprise TLS Security Proxy

What JA4proxy does

JA4proxy intercepts the TLS ClientHello — which is always transmitted in plaintext, before encryption begins. It computes a JA4 fingerprint from the cipher suites, extensions, ALPN, and elliptic curves advertised by each client, then routes the connection through a multi-signal scoring pipeline to decide: *allow, tarpit, block, or ban*. Allowed connections are forwarded byte-for-byte unchanged. The proxy never decrypts, never holds your private keys, and is transparent to both client and backend.

Key facts

- 0% false positive rate on browser traffic — by design
- 94–99% of malicious traffic blocked in testing
- Default dial = 0: monitor mode, never blocks on first deploy
- All bans carry a 5-minute TTL — false positives self-heal
- Never decrypts: no key custody, no SSL inspection compliance risk
- Scales horizontally; Redis synchronises bans across all instances

Resources

Source code & issue tracker: `github.com/seanpor/JA4proxy`

JA4 fingerprint specification: `github.com/FoxIO-LLC/ja4`

Documentation index: `docs/README.md` in the repository